applied sciences

MDPI

Article

Path Planning for Multi-Arm Manipulators Using Soft Actor-Critic Algorithm with Position Prediction of Moving Obstacles via LSTM

Kwan-Woo Park [1], MyeongSeop Kim [1,2], Jung-Su Kim [1,*] and Jae-Han Park [2]

[1] Research Center for Electrical and Information Technology, Department of Electrical and Information Engineering, Seoul National University of Science and Technology, Seoul 01811, Korea
[2] Applied Robot R&D Department, Korea Institute of Industrial Technology (KITECH), Ansan 15588, Korea
* Correspondence: jungsu@seoultech.ac.kr; Tel.: +82-2-970-6547

Abstract: This paper presents a deep reinforcement learning-based path planning algorithm for the multi-arm robot manipulator when there are both fixed and moving obstacles in the workspace. Considering the problem properties such as high dimensionality and continuous action, the proposed algorithm employs the SAC (soft actor-critic). Moreover, in order to predict explicitly the future position of the moving obstacle, LSTM (long short-term memory) is used. The SAC-based path planning algorithm is developed using the LSTM. In order to show the performance of the proposed algorithm, simulation results using GAZEBO and experimental results using real manipulators are presented. The simulation and experiment results show that the success ratio of path generation for arbitrary starting and goal points converges to 100%. It is also confirmed that the LSTM successfully predicts the future position of the obstacle.

Keywords: path planning; multi-arm manipulators; moving obstacles; reinforcement learning; soft actor-critic (SAC); hindsight experience replay (HER); collision avoidance; long short-term memory (LSTM)

check for updates

Citation: Park, K.-W.; Kim, M.; Kim, J.-S.; Park, J.-H. Path Planning for Multi-Arm Manipulators Using Soft Actor-Critic Algorithm with Position Prediction of Moving Obstacles via LSTM. *Appl. Sci.* **2022**, *12*, 9837. https://doi.org/10.3390/app12199837

Academic Editors: Yujin Lim and Hideyuki Takahashi

Received: 2 September 2022
Accepted: 27 September 2022
Published: 29 September 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the development of smart factories in the Fourth Industrial Revolution era, many robots are used in the manufacturing process [1–3]. Various types of robots work automatically within the workspace and perform repetitive tasks that are difficult for humans to do. Because it replaces manpower, robots have to cooperate like humans, and there should be no collisions in the process [4–7]. In this paper, we devise a path planning algorithm using deep reinforcement learning and deep learning in an environment with moving obstacles.

1.1. Background and Motivation

In a smart factory, robot manipulators perform various tasks in their workspace [8–10]. Most of the tasks performed by robots correspond to moving an object from a starting point to a goal point without collision. Therefore, it is important to create a path for the manipulator to move without colliding with other objects within the workspace. There are usually fixed obstacles and moving obstacles in a workspace. The fixed obstacle does not move while the robot is operating. So most path planning takes these fixed obstacles into account. In the case of only fixed obstacles, there are several path planning algorithms [11,12]. In industrial sites, a direct teaching method is used to design the path of a robot by an expert directly. The more interesting case is when there is a moving obstacle in the workspace. In this paper, we consider the case where an obstacle moves along a designated path. This is similar to the situation in a real factory. Moreover, as mentioned before, robots need to collaborate with each other. Therefore, manipulators must treat each other as obstacles and

avoid collisions when moving together at the same time. In general, since many robots operate simultaneously in a factory, it is common for one robot to consider other robots' entire workspace a fixed obstacle. This makes the robots move very inefficiently and the path between two arbitrary points in the workspace longer.

In path planning, handling moving obstacles in a systematic way is a challenging problem. In a heuristic approach relying on experts, they have to predict and calculate how obstacles move when planning a path but it is almost impossible. A famous algorithm for path planning is a sampling-based algorithm, called PRM (probabilistic roadmap) [13]. In this algorithm, after sampling from the workspace, the sampled nodes are connected to find a collision-free path connecting the starting and goal points. For such an algorithm, when the number of joints or the robot increases, the amount of computation increases as the dimension of the state increases. Furthermore, if the obstacle moves at irregular speeds, the problem becomes even more difficult and the existing path planning can not solve the problem efficiently. This paper focuses on using both a deep reinforcement learning (SAC: soft actor-critic) algorithm and deep learning (LSTM: long short-term memory) network to solve the optimal path plan without collision using a multi-arm manipulator for the case where there are both fixed and moving obstacles in the workspace at the same time.

1.2. Related Work

Usually, it is assumed that the path planner knows all the information about the environment before path planning [14]. Among traditional path planning algorithms, the Dijkstra algorithm plans the path so that the sum of weights of the nodes between the starting point and the goal point is the smallest [15]. This method is inefficient as the number of nodes increases since the amount of required computation increases. The A^* algorithm is the improved Dijkstra algorithm [11]. This algorithm can only be used in a fixed environment, that is when the obstacles do not move. PRM is representative of the sampling-based algorithms. RRT (rapidly exploring random tree) is another popular sampling-based path planning algorithm [16]. Although there are many merits in RRT, it needs enough sampling and can not guarantee the optimality of the path.

Artificial intelligent-based path planning has been developed and overcome the weak points of existing results [17–20]. Recently, deep reinforcement learning (RL) is also used to devise path planning [21–24]. The advantage of deep reinforcement learning-based path planning is that it can create a smoother path over existing results, and successfully enables path planning even in dynamic environments. This is demonstrated in [22]. In order to obtain successful deep reinforcement learning, it is of the utmost importance to consider how to check for collisions. To this end, the OBB (oriented bounding box) is mainly used in path planning. Considering the situation in the real factory, a common case is that obstacles move along a predetermined path in the workspace. Hence, the path planning algorithms have to consider not only the fixed obstacle but also the moving obstacles.

In order to devise a path planning taking both kinds of obstacles into account, a deep reinforcement learning-based algorithm is proposed in [22]. In the algorithm in [22], there are two key elements leading to successful path planning results. The first is HER (hindsight experience replay) which helps the RL algorithm overcome the sparse reward property of path planning. Note that since path planning is a typical sparse reward problem in terms of RL, it is not easy to train the deep RL agent. The second is that, in addition to the joint values of the manipulator, the current position of the moving obstacle is used as a part of the state. Hence, the current position of the moving obstacle is also used as the input to the neural network for RL-based path planning. In [22], such RL-based path planning can deal with the moving obstacle successfully. The rationale behind the RL-based path planning in [22] is that the neural network for the RL-based path planning agent computes the collision-free path by estimating the future position of the moving obstacle considering the current position of the moving obstacle. In other words, the future position of the moving obstacle is implicitly computed. In the algorithm in [22], it is not possible to check whether the RL-based path planning correctly estimates the future position of the moving

obstacle or not. This paper intends to overcome this weak point of the previous result by employing the LSTM in order to predict the future position of the moving obstacle explicitly. It is shown that the prediction by the LSTM based on the current and past obstacle points predicts the future position of the obstacle well and leads to successful path planning.

1.3. Proposed Method

For the multi-arm robot manipulator, this paper proposes a deep RL-based path planning algorithm capable of considering both fixed and moving obstacles. To this end, first, an MDP (Markov decision process) is defined properly for path planning of the multi-arm robot manipulator with both fixed and moving obstacles. Second, the LSTM is designed such that it can generate the prediction of the position of the moving obstacle over several future sample times using current and past position information. In order to train the LSTM, the training data can be obtained from the simulation or experiment. Third, based on the LSTM, the SAC-based path planning algorithm is developed such that the SAC agent uses not only the current joint values of the multi-arm robot manipulator but also the prediction of the obstacle from the trained LSTM. Since the SAC is known to have very efficient exploration performance for high dimensional MDP, it is employed together with HER for the proposed path planning. For both simulation and experiment, it is shown that the proposed SAC-based path planning generates the optimal path successfully connecting arbitrary given starting and goal points.

The workspace used in the case study of the proposed path planning in the simulation has obstacles moving at random speeds following a zigzag-shape path in the workspace, and a long fixed obstacle on the z-axis where there is overlap between the workspaces of the two manipulators. Moreover, in the experimental environment, the workspace has a fixed obstacle and an obstacle moving along the y-axis. The proposed algorithm is concerned with the same path planning problem as that in [22]. In the case of [22], path planning was carried out by receiving the position data of the obstacle directly from the environment. However, since the proposed method provides an explicit prediction of the future position of the obstacle using the LSTM, the proposed method is helpful for the application in which the explicit prediction information is required.

2. Background Concept and Problem Modeling

2.1. Path Planning for Robot Manipulator and Configuration Space

In this paper, configuration space $\mathbf{Q}$ is defined as the set of all joint angles of the manipulator. $\mathbf{Q}$ is a subset of $\mathbb{R}^n$, where n denotes the number of joints of the manipulator. Therefore, the position of the robot can be expressed as a point $q \in \mathbb{R}^n$ using values of n joint angle. q_i represents the ith joint angle value, and it holds $q = (q_1, q_2 \cdots q_n)$. Configuration space $\mathbf{Q}$ can be represented by two subsets. The first is defined as $\mathbf{Q}_{\text{free}}$ in which the robot does not collide with obstacles, itself, or other robots. The second is the space that is the complement of $\mathbf{Q}_{\text{free}}$, and it is denoted by $\mathbf{Q}_{\text{collide}}$ in which collision occurs. In other words, if the robot's position exists in $\mathbf{Q}_{\text{collide}}$, it means that the robot collides with an obstacle, a wall, another robot, or itself [5,25,26]. The robot manipulator does not collide if the joint angle belongs to $\mathbf{Q}_{\text{free}}$.

In the existing sampling-based method, random sampling points at $\mathbf{Q}_{\text{free}}$ are made and connected to form the shortest line, and this process is repeated to construct a path from a starting point to a goal point. Such lines and points constructed in this way are collision-free paths because they are in $\mathbf{Q}_{\text{free}}$. In addition, in this paper, $q_t \in \mathbf{Q} \in \mathbb{R}^n$ represents the joint angle of the robot manipulator at the tth time step, and T represents the maximum number of time steps during one episode of the algorithm. Therefore, when the episode starts, the algorithm must start at the starting point $q_{\text{init}} \in \mathbf{Q}_{\text{free}}$ and find the shortest possible collision-free path within the maximum number of steps, T, to arrive at the goal point $q_{\text{goal}} \in \mathbf{Q}_{\text{free}}$.

Finally, in the proposed algorithm, when the starting point $q_{\text{init}} \in \mathbf{Q}_{\text{free}}$ and goal point $q_{\text{goal}} \in \mathbf{Q}_{\text{free}}$ are input, the trained RL-based path planning algorithm calculates the shortest

path without collision. Hence, the path is defined as a sequence of states using q_t. In this case, for any $T_1 \leq T$, all elements in the path including the sequence $q_0 = q_{\text{init}}, \cdots, q_{T_1} = q_{\text{goal}}$ must belong to $\mathbf{Q}_{\text{free}}$ because there must be no collision.

2.2. Collision Detection in Workspace Using the Oriented Bounding Box (OBB)

The workspace denoted by $\mathbf{W} \in \mathbb{R}^3$ is defined as a space in which the robot manipulator works. In order for the robot manipulator to move in the workspace, it is necessary to check if it is guaranteed $q_t \in \mathbf{Q}_{\text{free}}$. To check this, collision detection methods are used [25,27]. In this paper, the OBB (oriented bounding box) [28] is employed. The OBB is a more accurate collision detection method than the AABB (axis aligned bounding box) [29]. The OBB method models objects (i.e., possible obstacles) in the workspace in the form of a rectangular box. Then, a collision test between the manipulator and an obstacle is performed using the two boxes of modeled objects. It is shown in [30] that a total of 15 calculations are performed using each of the three faces of the two boxes that use collision detection and the nine edges between the two boxes. Therefore, each box can be expressed as a robot manipulator and an obstacle, and collisions with all obstacles are detected using the OBB method repeatedly [31]. If all these steps are passed, it can be sure that collision is free, i.e., $q_t \in \mathbf{Q}_{\text{free}}$.

2.3. Reinforcement Learning

Reinforcement learning aims to find an answer to a given MDP (Markov decision process) problem. MDP refers to the process by which an agent interacts with the environment through actions using the current state, and obtains a reward. As a basic method of solving such an MDP, an optimization-based method is used in which an agent interacts with a probabilistic environment, selects an action, and maximizes the sum of rewards [32]. MDP is usually defined in the form of $\{S, A, P, r, \gamma\}$. S is the state set, A is the action set, P denotes the state transition probability, $r(s, a)$ is the reward function, and γ is the discount factor [33]. The state transition probability $P(s'|s, a)$ represents the probability of transition to the next state s' when the action $a \in A$ is taken from the current state s. The policy $\pi(a|s)$ is the rule that determines how the agent interacts with the environment when it is in the current state s. A policy can be expressed as a probability distribution or a deterministic function. The agent interacts with the environment at every time step. The agent selects the action $a_t \in A$ according to the policy $\pi : S \rightarrow A$ and the state $s_t \in S$ and injects it into the environment. The environment returns the next state $s_{t+1} \in S$ and the reward r_{t+1} to the agent according to the state transition probability $P : S \times A \rightarrow [0, 1]$ based on the current state and injected action. The purpose of reinforcement learning is to use this procedure to iteratively update the policy $\pi(a|s)$ so that the agent can receive the maximum reward sum until it approaches the optimal policy π^*. That is, if the policy is updated with the optimal policy, $E_{\pi(a|s)}[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}] = E_{\pi^*}[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}]$ is established. There are two main ways to update the optimal policy. The first is value-based methods that update the optimal value function (i.e., the function that estimates the expected value of the reward sum) such as DQN (deep Q-network), DRQN (deep recurrent Q-learning for partially observable MDPs) using a neural network-like function approximation [34,35]. The action is chosen based on the value function. The second method, called the policy gradient method, learns the optimal policy directly from the agent's experience. Policy-based learning is a method of determining the action a directly using the state s, rather than using a value function to determine the action. Representative methods of policy-based learning are REINFORCE, the actor-critic method, DPG (deterministic policy gradient), DDPG (deep DPG), A3C (asynchronous advantage actor-critic), TRPO (trust region policy optimization), PPO (proximal policy optimization), MPO (maximum a posteriori policy optimization), D4PG (distributed distributional DDPG), etc. [36–43]. Reinforcement learning mostly uses a mini-batch method for learning. In order to perform batch learning, we need to collect many data samples. A sample usually consists of a state s_t, an action a_t, a reward r_{t+1}, and the next state s_{t+1}. Moreover, many reinforcement learning algorithms rely on techniques that

use samples in various specific ways for efficient learning. Typically, the replay buffer, PER (prioritized experience replay), and HER are mainly used [34,44,45]. This paper employs SAC as a reinforcement learning algorithm with HER for path planning of a multi-arm manipulator with both fixed and moving obstacles [46].

2.4. Long Short-Term Memory (LSTM)

Since it is difficult for the RNN (recurrent neural network) to remember the input information when there is a large time difference between output and input information (called the long-term dependence problem), the LSTM is devised to overcome this problem. To solve this disadvantage, the LSTM uses three gates and modifies information by adding a cell state to the internal structure of the neural network. More specifically, the LSTM has three main gates. The first is a forget gate, which determines how much information is discarded and stored in the cell state. The second is the input gate, which decides how much of the new information coming to the input gate is to be stored in the cell state. The third is the output gate, which determines how much information the cell state has to output. It has the ability to preserve long-term memory by using the gates that are not in RNNs. Therefore, the LSTM is mainly used for time series data processing or natural language processing [47–50]. In this paper, the LSTM is used to predict the future position of a moving obstacle, and the SAC agent uses the predicted position of the moving obstacle for the proposed SAC-based path planning.

3. Soft Actor-Critic Based Path Planning for Moving Obstacles with LSTM

This section explains the main result of the paper, i.e., SAC-based path planning algorithm for multi-arm robot manipulator considering both fixed and moving obstacles. To this end, first, MDP is properly defined and then it is explained how to design the LSTM for the prediction of the future position of the moving obstacle. Finally, the structure of the proposed SAC-based path planning and case study are presented. Note that the existing result in [22] was concerned with a similar path planning problem. Since this paper intends to propose a more useful solution to the problem considered in [22], the notations and problem definition in this paper are inevitably similar to those in [22].

3.1. Path Planning for the Multi-Arm Manipulator and Augmented Configuration Space

For the purpose of establishing MDP, the configuration space is defined first. Since there are n joints of each robot arm, the degree of freedom (DOF) of the robot arm manipulator is n, and it is assumed that there are m robot arm manipulators. Therefore, as defined above, $q_t^i \in \mathbb{R}^n$ means the set of n joint angle values of the ith robot arm at the tth time step of the SAC training samples. Expanding this, $q_{t,j}^i \in \mathbb{R}^n$ is a component of $q_t^i \in \mathbb{R}^n$, which means the jth joint angle value of the ith robot arm at the tth time step. It is difficult to define the configuration space for the path planning algorithm of the multi-arm manipulator in an environment with time-varying obstacles. This is because the manipulators influence each other on the definitions of $\mathbf{Q}_{\text{free}}^i$ and $\mathbf{Q}_{\text{collide}}^i$ that define the configuration space of the manipulator because $\mathbf{Q}_{\text{free}}^i$ changes every moment while the manipulator is operating. In other words, it can be viewed as an environment in which there are more obstacles that do not know how to move in addition to the fixed and moving obstacles. Therefore, it is recommended to define multiple manipulators as one virtual manipulator in order to make the definition of the configuration space easier [51,52]. Namely, we define a multi-arm manipulator as a single virtual manipulator with $n \times m$ joints. The new configuration space is as follows:

$$q_t = \begin{bmatrix} q_t^1 \\ q_t^2 \\ \vdots \\ q_t^m \end{bmatrix} \in \mathbb{R}^{nm}, \quad q_t^i = \begin{bmatrix} q_{t,1}^i \\ q_{t,2}^i \\ \vdots \\ q_{t,n}^i \end{bmatrix} \in \mathbb{R}^n, \quad i \in \{1, 2, \cdots, m\}. \tag{1}$$

Therefore, the newly defined augmented configuration space is expressed as $\mathbf{Q}^a := \mathbf{Q}^a_{\text{free}} \bigcup \mathbf{Q}^a_{\text{collide}}$ and is as follows.

$$\mathbf{Q}^a_{\text{free}} = \underbrace{\mathbf{Q}^1_{\text{free}} \times \cdots \times \mathbf{Q}^n_{\text{free}}}_{m \text{ times}} \subset \mathbb{R}^{nm} \text{ and } \mathbf{Q}^a_{\text{collide}} = \underbrace{\mathbf{Q}^1_{\text{collide}} \times \cdots \times \mathbf{Q}^n_{\text{collide}}}_{m \text{ times}} \subset \mathbb{R}^{nm},$$

In the expression, $\mathbf{Q}^a_{\text{free}}$ means the space where there is no collision in the augmented configuration space $\mathbf{Q}^a$, and $\mathbf{Q}^a_{\text{collide}}$ means the space where the collision occurs in augmented the configuration space $\mathbf{Q}^a$. In this case, it is necessary to reinterpret the path planning problem of the multi-arm manipulator using an augmented configuration space. We redefine it as a path planning problem for a single virtual manipulator has the start point $q_0 \in \mathbf{Q}^a_{\text{free}}$ and the goal point $q_{\text{goal}} \in \mathbf{Q}^a_{\text{free}}$ in the augmented configuration space.

$$q_0 = \begin{bmatrix} q^1_{\text{init}} \\ q^2_{\text{init}} \\ \vdots \\ q^m_{\text{init}} \end{bmatrix}, \quad q_{\text{goal}} = \begin{bmatrix} q^1_{\text{goal}} \\ q^2_{\text{goal}} \\ \vdots \\ q^m_{\text{goal}} \end{bmatrix}.$$

In the expression, q^i_{init} and q^i_{goal} mean the starting point and goal point of the ith robot manipulator.

3.2. Multi-Arm Manipulator MDP with Position Estimation of Moving Obstacles via LSTM

Since the path planning algorithm of the multi-arm manipulator using LSTM (MAMMDP$_{\text{mo}}$) proposed in this paper uses deep reinforcement learning, it is necessary to define the tuple $\{S, A, P, r, \gamma\}$ for the multi-arm manipulator MDP. Figure 1 describes the structure of MAMMDP$_{\text{mo}}$.

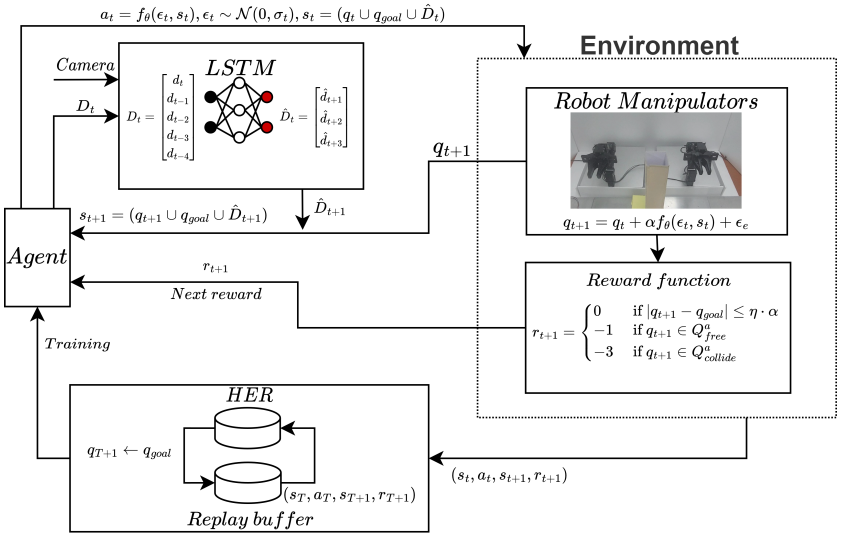


Figure 1. Multi -Arm Manipulator MDP with Position Estimation of Moving Obstacles via LSTM (MAMMDP$_{\text{mo}}$).

The MAMMDP$_{\text{mo}}$ proposed in this paper is similar to the MAMMDP of [22]. The difference is that in [22], information about the obstacles (i.e., past and current positions) is obtained directly from the environment and it is a part of the state. On the other hand, in the proposed method, the agent separately predicts the future position of moving obstacles using the trained LSTM with past and current positions and uses the predicted position for SAC-based path planning.

To this end, the LSTM is trained using the measured position data of the moving obstacle in advance considering the work environment and moving obstacles before the

training of the SAC-based path planning. In practice, position information can be obtained through various sensors (e.g., cameras). At every sampling time, the position data of the obstacle are acquired and stored using machine vision technology. For the training of the LSTM, the input of the LSTM is the obstacle position over the past n_p sampling times, and the output of the LSTM is the prediction of the obstacle position for the future n_f sampling times. n_p denotes the number of time steps for the obstacle's past position information, and n_f indicates how far the LSTM predicts the position of the obstacle after the current time point. As n_f increases, the LSTM's output dimension, i.e., the agent's state dimension becomes large, which can make the training of SAC-based path planning computationally demanding. Hence, an appropriate tradeoff is required. Figure 2 shows the I/O (input/output) structure of the LSTM, and the I/O variables of the LSTM are defined as follows.

$$D_t = \begin{bmatrix} d_t \\ d_{t-1} \\ \vdots \\ d_{t-(n_p-1)} \end{bmatrix} \in \mathbb{R}^{3 \cdot n_p}, \quad \hat{D}_t = \begin{bmatrix} \hat{d}_{t+1} \\ \hat{d}_{t+2} \\ \vdots \\ \hat{d}_{t+(n_f)} \end{bmatrix} \in \mathbb{R}^{3 \cdot n_f}, \quad D_{target,t} = \begin{bmatrix} d_{t+1} \\ d_{t+2} \\ \vdots \\ d_{t+(n_f)} \end{bmatrix} \in \mathbb{R}^{3 \cdot n_f}$$

$$d_t = \begin{bmatrix} x_t^d \\ y_t^d \\ z_t^d \end{bmatrix} \in \mathbb{R}^3, \quad \hat{d}_t = \begin{bmatrix} \hat{x}_t^d \\ \hat{y}_t^d \\ \hat{z}_t^d \end{bmatrix} \in \mathbb{R}^3,$$

where $d_t = \begin{bmatrix} x_t^d & y_t^d & z_t^d \end{bmatrix}^T$ and $\hat{d}_t = \begin{bmatrix} \hat{x}_t^d & \hat{y}_t^d & \hat{z}_t^d \end{bmatrix}^T$ denote the position of the obstacle in the workspace and its prediction at the sampling time t, respectively. Therefore, the state at sampling time t is defined as $s_t = (q_t \cup q_{goal} \cup \hat{D}_t)$. In other words, for accurate path planning, the state includes not only the current joint information of the multi-arm manipulator but also the prediction of the future position of the obstacle using $\hat{D}_t$.

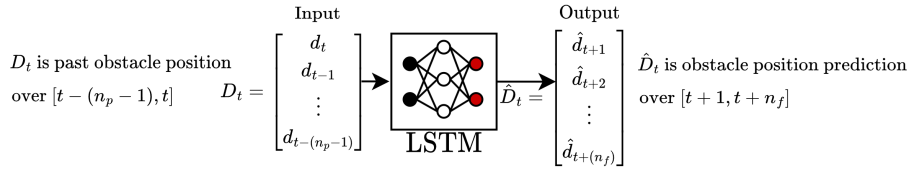


Figure 2. Architecture of LSTM network for position prediction.

The LSTM is trained using the following cost function

$$Loss = \frac{1}{N} \Sigma_{t=1}^{N} (D_{target,t} - \hat{D}_t)^2, \tag{2}$$

where $D_{target,t}$ denotes the true position of the moving obstacle, and $\hat{D}_t$ is the output of the LSTM, meaning the prediction of the position of the moving obstacle. As a matter of fact, there are two possibilities to obtain the current position d_t of the moving obstacle for the LSTM training. The first is that the position of the moving obstacle can be obtained as a time series using a sensor such as a camera. To be specific, after running only the moving obstacle without robot manipulators, the camera gets the position information and saves them in a buffer. Then, the LSTM can be trained using the time series. The second is that the position d_t can be made using simulation. If the trajectory followed by the moving obstacle is known, that information can be used as the time series for the LSTM training. To obtain the training data of the LSTM in this paper, the robot arm does not move in the simulation, but only the obstacle moves within the workspace. The SAC-based path planning agent receives the position of the obstacle from the environment at every step. The

received obstacle position data are stored in a buffer and the training data D_t and $D_{target,t}$ is generated from the buffer.

The action is defined as $a_t = f_\theta(\epsilon_t, s_t)$. Here, ϵ_t is an added noise in order to model the uncertainty of the environment and assumed to follow the Gaussian distribution $\mathcal{N}(0, \sigma_t)$ where σ_t represents variance. $f_\theta(\epsilon_t, s_t)$ denotes the output of the policy network when the input to the policy network is s_t. Consequently, a_t can be viewed as a sampled value from a Gaussian distribution. Then, the state at the next time step is calculated using the current state value and the relation $\hat{q}_{t+1} = q_t + \alpha a_t + \epsilon_e$, where the tuning parameter α is defined as the geometric mean of the maximum variation of each joint and ϵ_e is the environmental noise following $\epsilon_e \sim \mathcal{N}(0, \sigma_e)$. If the next state value causes a collision after applying the action, that is, if it is $\hat{q}_{t+1} \in \mathbf{Q}^a_{\text{collide}}$, the state does not move and maintains the current state value. The following summarizes how the joint values are updated during path planning.

$$q_{t+1} = \begin{cases} \hat{q}_{t+1}, & \text{if } \hat{q}_{t+1} \in \mathbf{Q}^a_{\text{free}}, \\ q_t, & \text{if } \hat{q}_{t+1} \notin \mathbf{Q}^a_{\text{free}}. \end{cases}$$

Using this, the next state is expressed as $s_{t+1} = (q_{t+1} \cup \hat{D}_{t+1})$, where $\hat{D}_{t+1}$ is the prediction of the obstacle position by the LSTM after receiving q_{t+1} from the environment. Until the joint value included in the next state reaches goal point q_{goal}, the process of applying the action according to the policy and obtaining the state and reward is repeated. The final goal is given by $q_{t+1} = q_{\text{goal}}$. However, this is very difficult due to practical limitations such as numerical errors. Therefore, using the norm of the vector, the final goal is changed as $\|q_{t+1} - q_{\text{goal}}\| \leq \eta \cdot \alpha$. Here, $\|\cdot\|$ means the norm of the vector, and α is the maximum action step such that the manipulator does not move much at once. η is a tuning variable that satisfies $0 < \eta < 1$. That is, the goal of learning is set to arrive near the goal point, rather than finding the perfect goal point such as $q_{t+1} = q_{\text{goal}}$. The following is a reward function $r(s_t, a_t)$ used for learning.

$$r_{t+1} = \begin{cases} 0, & \text{if } |q_{t+1} - q_{\text{goal}}| \leq \eta \cdot \alpha \\ -1, & \text{if } q_{t+1} \in \mathbf{Q}^a_{\text{free}} \\ -3, & \text{if } q_{t+1} \in \mathbf{Q}^a_{\text{collide}} \end{cases} \tag{3}$$

This gives a greater penalty to collisions with obstacles, helping the agent learn to avoid collisions as much as possible. Therefore, if the agent does not reach the goal point by the maximum number of iterations T, then the minimum total reward received is $-3 \times T$. However, if it arrives at the goal point in a specific iteration $T_{done} < T$, the reward is better than that.

In the next section, we present a method to update the policy during training using a deep neural network.

3.3. Soft Actor-Critic (SAC)-Based Path Planning with Prediction of Moving Obstacle Position via LSTM

This section briefly reviews how the SAC is used for the proposed SAC-based path planning. The proposed MAMMDP$_{\text{mo}}$ has a continuous action space. There are many algorithms for MDP with continuous action in reinforcement learning [38,41,46,53]. This paper employs the SAC, which is known to have good performance for high dimensional MDP with continuous action such as robotics. While other algorithms usually maximize only the sum of the expected rewards, SAC maximizes the sum of the value function and entropy. If the existing algorithms are updated in a way that maximizes the sum of the expected rewards, the policy π is made such that a specific action is increased. However, considering the policy entropy in the SAC helps the agent to explore a wide range of various actions. In the SAC, soft state value function V and soft action value function Q are defined as follows.

$$V(s_t) = E_{a_t}[Q(s_t, a_t) - \beta log \pi(a_t|s_t)], \tag{4}$$

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma E_{s_{t+1}}[V(s_{t+1})], \tag{5}$$

where β means a temperature parameter that controls the weight for Q function and the entropy term $log\pi(a_t|s_t)$. The Equation (4) is called a soft state function used in the SAC. Since the entropy term needs to be maximized, the gradient ascent should be used. So the sign is negative. The Equation (5) is a soft state-action function used in SAC. If the action is assumed to be finite, repeatedly executing the soft Bellman equation, Equation (5), with a given policy leads to a soft Q-function. In order to estimate the functions, the SAC uses neural networks. To create training data for the SAC networks, $\hat{D}_t$ is obtained using the trained LSTM network using D_t as an input. Using this, we created a state s_t, obtained a_t, and then interacted with the environment to store training data $\{s_t, a_t, s_{t+1}, r_{t+1}\}$ in replay buffer at each step. Specifically, the SAC utilizes five neural networks to estimate the policy, state value, and state-action value functions; see Figure 3.

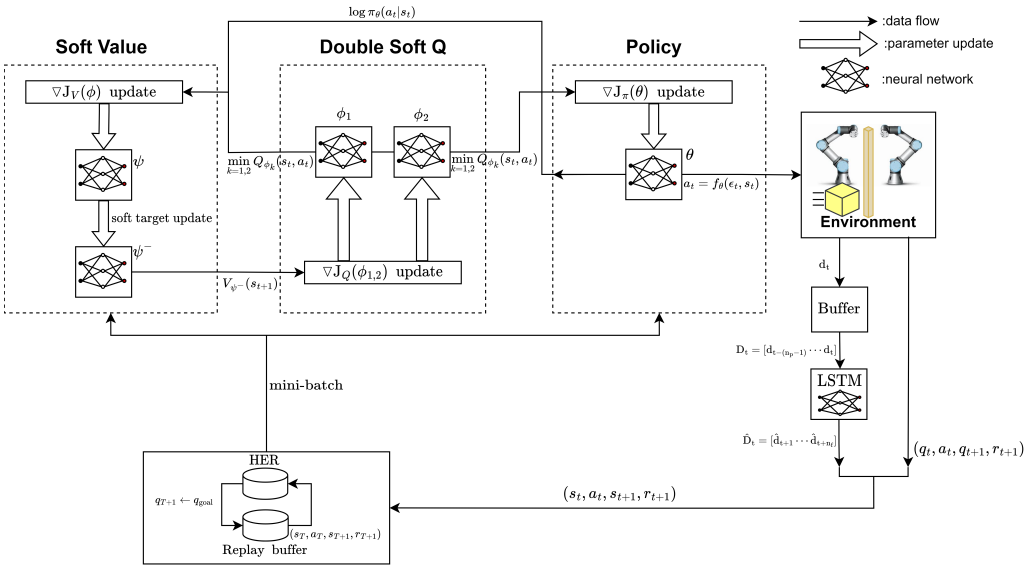


Figure 3. Architecture of the proposed SAC-based path planning algorithm with HER.

In order to estimate the state value function V, in the proposed algorithm, the value network V_ψ and its target network V_{ψ^-} are used. The target network V_{ψ^-} is used to improve training performance for the Q-network. The subscripts ψ and ψ^- indicate the parameters of the neural networks that approximate the functions. The cost function (6) is used to update the parameters ψ of the neural network for the value function.

$$J_V(\psi) = E_{s_t}[\frac{1}{2}(V_\psi(s_t) - [min_{k=1,2}Q_{\phi_k}(s_t, a_t) - \beta log\pi_\theta(a_t|s_t)])^2] \tag{6}$$

To prevent sudden change, the target network is used. Moreover, the target network has to change smoothly. Hence, parameter update is performed based on (7)

$$\psi^- = \tau\psi + (1-\tau)\psi^- \tag{7}$$

where the parameter $\tau \in [0, 1]$ controls the learning speed.

Two neural networks Q_{ϕ_1} and Q_{ϕ_2} approximate the Q-function and the minimum value of the two values is selected for learning. This helps to prevent Q values from being inappropriately overestimated [53]. The cost function (8) is minimized to update the parameters of Q_{ϕ_1} and Q_{ϕ_2}.

$$J_Q(\phi_{k=1,2}) = E_{(s_t,a_t)}[\frac{1}{2}(Q_{\phi_{k=1,2}}(s_t, a_t) - (r(s_t, a_t) + \gamma[V_{\psi^-}(s_{t+1})]))^2] \tag{8}$$

Similarly, the neural network parameterized by θ approximates the policy by minimizing the cost function (9).

$$J_\pi(\theta) = E_{s_t}[\beta \log \pi_\theta(a_t|s_t) - min_{k=1,2} Q_{\phi_k}(s_t, a_t)] \quad (9)$$

Since the proposed MAMMDP$_{\mathrm{mo}}$ has continuous action space, the output of the policy network is the standard deviation and mean for a Gaussian distribution and the action is sampled from the distribution.

As described in [22], MAMMDP$_{\mathrm{mo}}$ has a sparse reward, which makes it difficult for the SAC agent to be trained. To overcome this, HER is used [45]. For details, please see [22,45].

Figure 3 describes the detailed structure of the SAC-based path planning algorithm proposed in this paper. The structure is very similar to the structure proposed by [22] but the position prediction by the LSTM is embedded in the SAC-based path planning.

When the SAC-based path planning agent is successfully trained, the trained agent can generate the optimal path linking given arbitrary starting and goal points. Figure 4 describes how the trained SAC-based path planning agent generates the path. In other words, with the given starting point and the information on the position of the moving obstacle, the action a_t is computed by the trained policy network and is applied to the robot. Then, the real robot changes each joint. The camera installed on top of the workspace measures the current position of the moving obstacle using the OpenCV library, and stores the measurements in a buffer. The trained LSTM uses the data within the buffer to calculate the prediction of the future position of the moving obstacle. Finally, the state including the current joint information and the prediction of the moving obstacle is given to the trained policy network and the procedure is repeated until the state reaches the goal point.

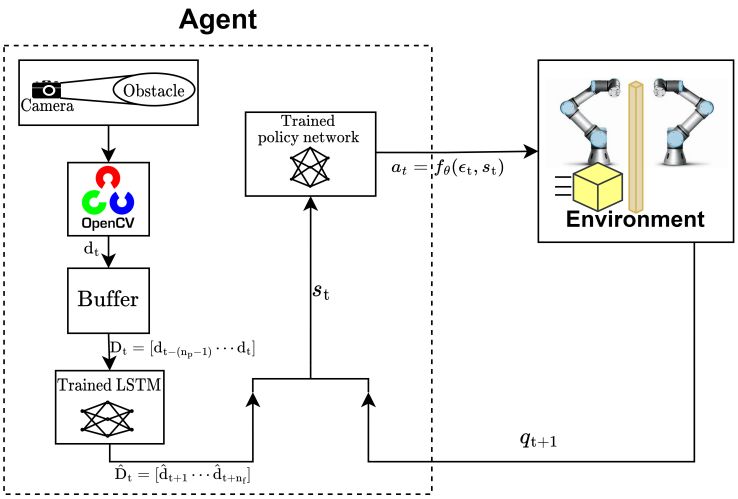


Figure 4. The trained SAC-based path planning agent for testing.

Algorithm 1 at the end of the paper summarizes the proposed SAC-based path planning with the LSTM.

Algorithm 1 Proposed SAC-based path planning algorithm for multi-arm manipulators.

for $e = 1$ `to` M **do** ▷ Train LSTM network
  Initialize local buffer N and 5-size buffer D_t
  $d_t, D_{target,t}$ = env.step() ▷ Collect the position data
  Stack d_t in D_t
  $\hat{D}_t$= LSTM(D_t)
  Store ($\hat{D}_t$,$D_{target,t}$) in N
  Sample mini-batch of m ($\hat{D}_t$,$D_{target,t}$) from N
    $J = \frac{1}{m}\sum_{i=1}^{m}(\hat{D}_i - D_{target,i})^2$
  LSTM network is updated by gradient descent using ∇J
end for

Define MAMMDP$_{\mathrm{mo}}$ and the current state $s_t = (q_t \cup q_{\mathrm{goal}} \cup D_t)$ and q_{init} and q_{goal}
Initialize network parameters $\psi, \phi_{1,2}, \theta$
Initialize the parameter values of the target network $\bar{\psi} \leftarrow \psi$
Initialize global replay memory $\mathcal{K}$
Initialize environment and get D_t

for $e = 1$ `to` M **do**
  Initialize local buffer $\mathcal{L}$ ▷ Memory for an episode
  for $t = 0$ `to` $T - 1$ **do**
    Get future position of obstacle with LSTM and the goal and initial positions $q_{\mathrm{goal}}, q_{\mathrm{init}} \in \mathbf{Q}^a_{\mathrm{free}}$
    $\hat{D}_t$= LSTM(D_t), $s_t = (q_t \cup q_{\mathrm{goal}} \cup \hat{D}_t)$, $a_t = f_\phi(\epsilon_t, s_t)$ $\epsilon_t \sim \mathcal{N}(0, \sigma_t)$
    $q_{t+1}, d_{t+1}, r_{t+1}$ = env.step(a_t)
    Stack d_{t+1} in D_{t+1}
    $\hat{D}_{t+1}$= LSTM(D_{t+1})
    $s_{t+1} = (q_{t+1} \cup q_{\mathrm{goal}} \cup \hat{D}_{t+1})$

    Store the transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{K}, \mathcal{L}$
    ▷ Parameters update
    Sample mini-batch of m transitions $(s_l, a_l, r_{l+1}, s_{l+1})$ from $\mathcal{K}$
      $J_V(\psi) = \mathbb{E}_{s_l}[\frac{1}{2}(V_\psi(s_l) - \mathbb{E}_{a_l}[\min_{k=1,2} Q_{\phi_k}(s_l, a_l) - \beta \log \pi_\theta(a_l|\, s_l)])^2]$
      $J_Q(\phi_{k=1,2}) = \mathbb{E}_{s_l,a_l}[\frac{1}{2}(Q_{\phi_{k=1,2}}(s_l, a_l) - (r_{l+1} + V_{\bar{\psi}}(s_{l+1})))^2]$
      $J_\pi(\theta) = \mathbb{E}_{s_l,a_l}[\beta \log \pi_\theta(a_l|s_l) - \min_{k=1,2} Q_{\phi_k}(s_l, a_l)]$

    Each network parameters $\psi, \theta_{1,2}, \phi$ are updated by gradient descent
    using $\nabla_\psi J_V(\psi), \nabla_{\phi_1} J_Q(\phi_1), \nabla_{\phi_2} J_Q(\phi_2), \nabla_\theta J_\pi(\theta)$

    Update state value target $\bar{\psi} \leftarrow \tau\psi + (1 - \tau)\bar{\psi}$
  end for

  if $q_T \neq q_{\mathrm{goal}}$ **then** ▷ HER
    for $t = 0$ `to` $T - 1$ **do**
      Sample a transition $(s_t, a_t, r_{t+1}, s_{t+1})$ from $\mathcal{L}$
      Change goal point $q'_{\mathrm{goal}} = q_{t+1}$
      if $q_{t+1} \in Q^a_{collide}$ **then**
        continue
      else if $|q_{t+1} - q'_{\mathrm{goal}}| \leq \eta \cdot \alpha$ **then**
        $r'_{t+1} = 0$
      end if
      Store the transition $(s_t, a_t, r'_{t+1}, s_{t+1})$ in $\mathcal{K}$
    end for
  end if
end for

4. Case Study

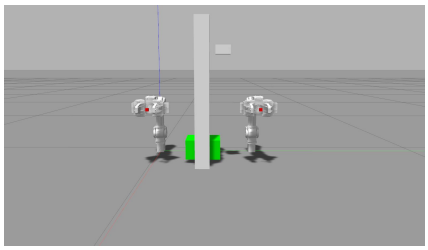
In this section, the SAC-based path planning for the multi-arm manipulator with a moving obstacle using the LSTM presented so far is implemented. To this end, two 3-DOF open manipulators are used to test the performance of the proposed SAC-based path planning algorithm. More information on the used manipulator can be found in https://url.kr/4dvxeq (accessed on 25 September 2022). The total workspace for the case study is 70 cm × 120 cm × 100 cm, and the size of the moving obstacle is 15 cm × 15 cm × 5 cm. Moreover, there are both fixed and moving obstacles. The parameters for the rest of the proposed algorithm are summarized in Table 1.

Table 1. Parameters of the 3-DOF manipulator and environment.

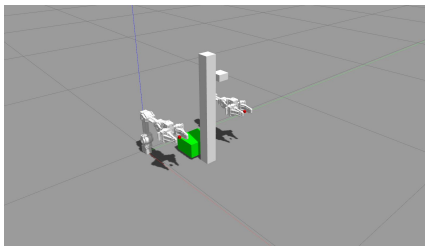
Name	Value	Notation	Unit
The number of manipulator joints	3	n	pc. (Piece)
The number of manipulators	2	m	pc. (Piece)
Joint maximum	(140, −45, 150, 140, −45, 150)		° (Degree)
Joint minimum	(−140, −180, 45, −140, −180, 45)		° (Degree)
Dimension of $\mathbf{Q}^a_{\text{free}}$	6	$n \cdot m$	Dimension
The number of dynamic obstacle	1		pc. (Piece)
The number of past features	5	n_p	Feature
The number of future features	3	n_f	Feature
The dynamic obstacle axis	x, y, z		

4.1. Simulation

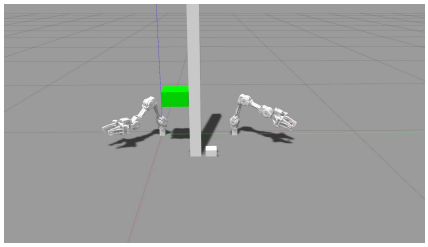
For the simulation-based case study, a moving obstacle is considered such that it follows a zigzag-shaped trajectory along a vertical line in the workspace repeatedly. Figure 5 depicts the considered workspace implemented with GAZEBO. In the GAZEBO simulation, dynamic objects such as obstacles and robot manipulators communicate with each other in the ROS (robot operating system). The moving obstacle is depicted in the form of green boxes, and the gray box indicates the fixed obstacle in Figure 5. Figure 5a,b are the initial setups of the simulation environment before running the proposed algorithm. Figure 5c,d show the snapshot of the simulation environment when the proposed algorithm is working.



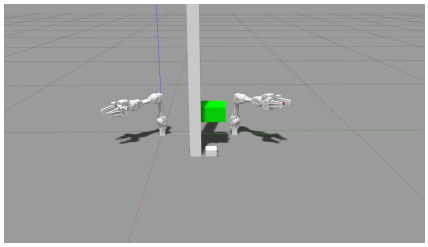
(**a**) Initial setup of the simulation environment



(**b**) Initial setup of the simulation environment



(**c**) The snapshot of the simulation environment



(**d**) The snapshot of the simulation environment

Figure 5. The GAZEBO workspace.

For the LSTM training, at first, without running the robot manipulators, by operating the moving obstacle only in GAZEBO, its position data are saved as a time series. Using the

time series data, the LSTM is trained using (2). Figure 6 demonstrates the loss function of the LSTM to show the training performance. In view of the convergence of the loss function, the LSTM is trained successfully.

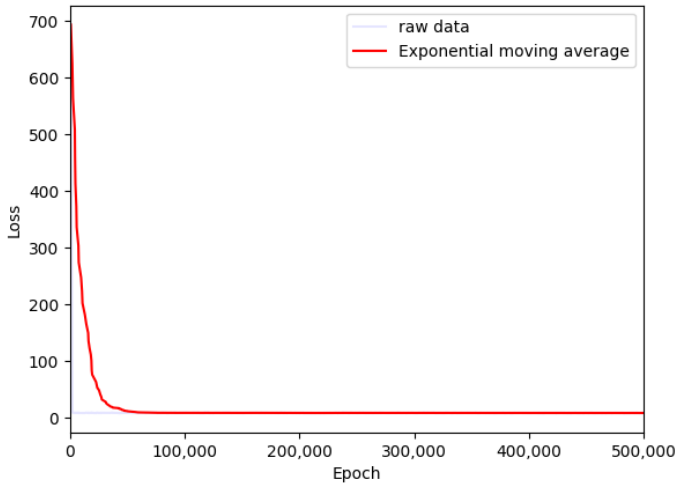


Figure 6. Loss function of simulation LSTM training.

Using the trained LSTM, which acts as the predictor of the position of the moving obstacle, the proposed SAC-based path planning agent is trained. It is assumed in the simulation-based case study that D_t is obtained from the environment and it is given to the trained LSTM in order to compute $\hat{D}_t$. The five neural networks for the proposed SAC-based path planning in Figure 3 are trained through 100,000 episodes using a Nvidia GeForce RTX 3080Ti GPU and AMD Ryzen 5 5600X CPU. The networks are implemented using Tensorflow 2.5. It takes 19 h for the training. In addition, for 100,000 episodes, random starting and goal points are used at the beginning of each episode. Figure 7 shows the success rate of path creation in the course of learning 100,000 episodes. Success rates are calculated for every 30 episodes. It can be seen that the success rate converges to about 95% as the learning progress is going on until 100,000 episodes.

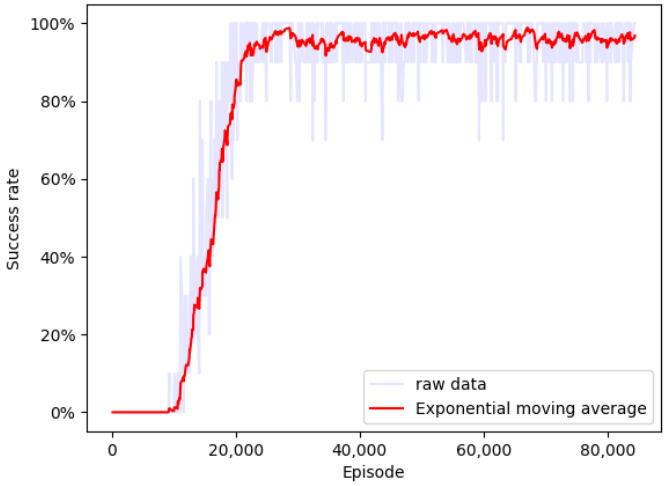


Figure 7. Simulation: success ratio of the proposed path planning in training.

For the evaluation of training performance, not only the success rate but also the reward matters. Figure 8 shows the reward while learning. One of the goals of this paper is to compute the shortest route. As the training goes on, the SAC agent finds shorter paths due to the definition of the reward function in (3). This is why the reward gets better as the training goes on in Figure 8.

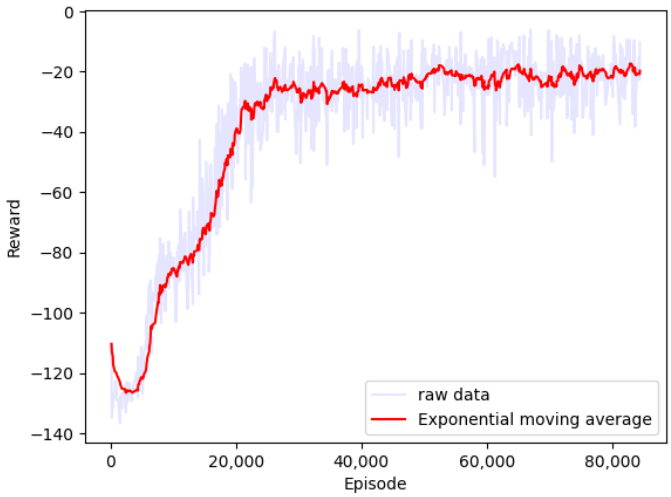


Figure 8. Simulation: reward of the proposed path planning in training.

To show the performance of the proposed algorithm, Figures 9 and 10 show the success ratio and reward, respectively. The success ratio means if the SAC agent generates a collision-free path or not when an arbitrary initial point and goal points are given. As the training goes on, the success ratio converges to 1. This means that the trained SAC agents compute a collision-free path better as the training goes on, and after some time, the SAC agent finds a collision-free path without fail. The reward graph has a similar meaning. The same interpretation is applied to the experimental results (see Figures 11 and 12). Additionally, a comparison is shown in Figure 9 with the existing result [22]. The training performance of the proposed method outperforms compared with [22] in terms of training speed when all hyperparameters are set to the same conditions. This shows that explicitly obtaining and using future obstacle predictions using LSTM leads to better performance.

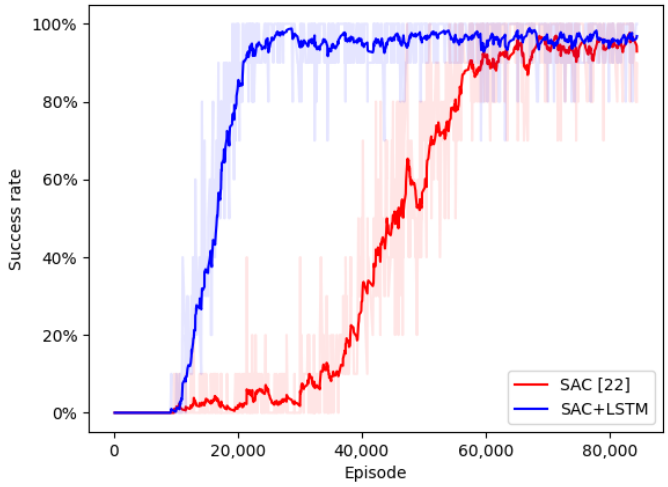


Figure 9. Comparison of success rates.

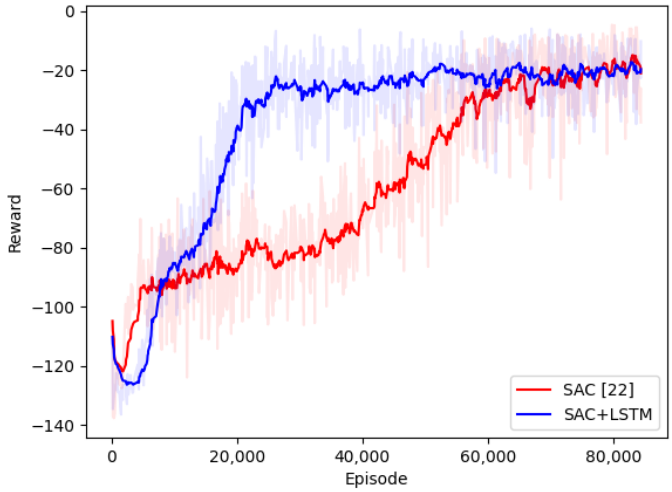


Figure 10. Comparison of reward.

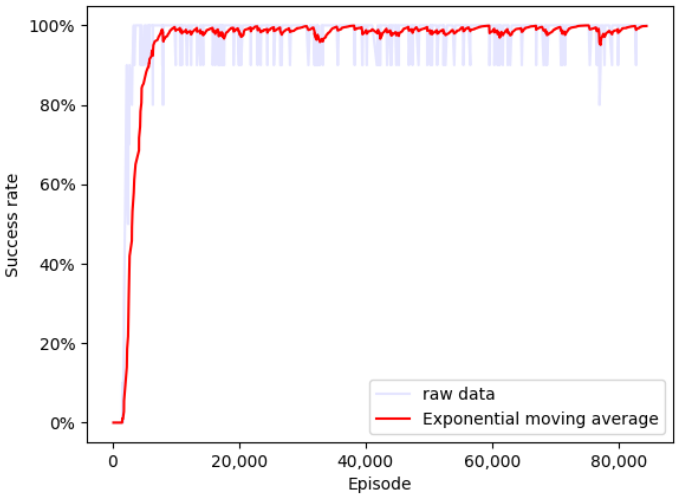


Figure 11. Experiment: success ratio of the proposed path planning in training.

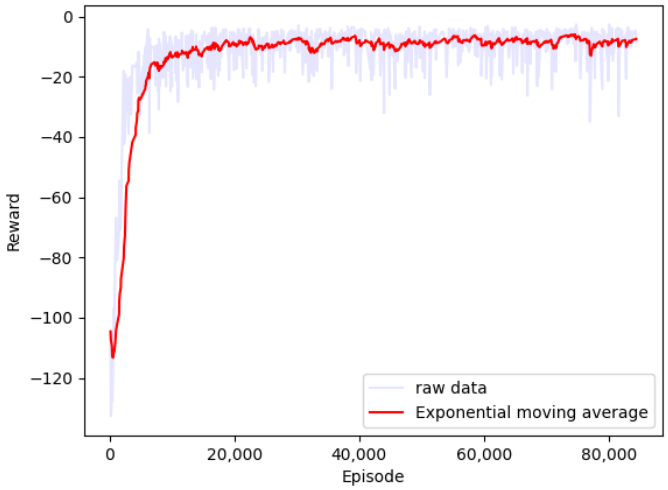
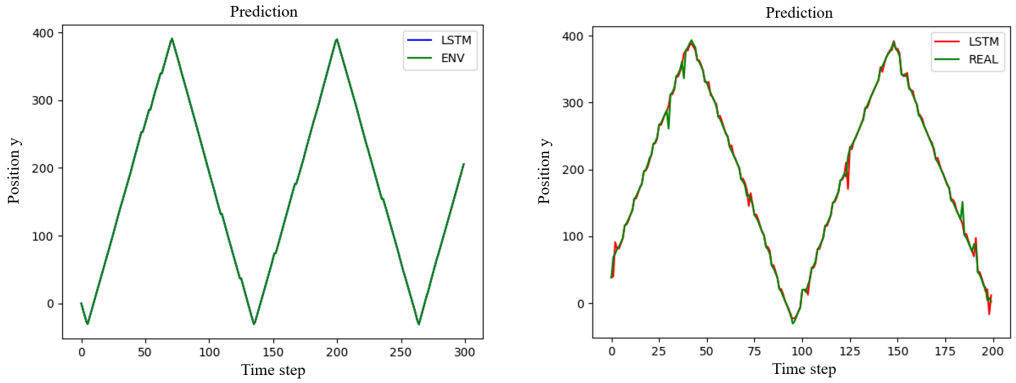


Figure 12. Experiment: reward of the proposed path planning in training.

In addition to the faster training speed in Figure 9, the reward also converges more quickly, as shown in Figure 10.

Since the SAC-based path planning is trained, its testing result can be obtained for arbitrary starting and goal points according to the procedure in Figure 4. The testing results are demonstrated on the web page https://url.kr/4m562t (accessed on 25 September 2022). Figure 13a shows that the trained LSTM predicts the future position of the moving obstacle successfully in the simulation-based case study. The prediction by the trained LSTM is compared with the real obstacle position by saving 300 test results in simulation. In Figure 13a, the average prediction error of 300 measurements is 0.08%.



(**a**) (**b**)

Figure 13. LSTM prediction results in simulation and experiment. (**a**) Simulation environment. (**b**) Experiment.

The hyper-parameters of the SAC-based path planning are presented in Table 2.

Table 2. Tuning parameters for the designed SAC with HER.

Name	Value	Notation	Unit
Policy network size	$12 \times 800 \times 500 \times 400 \times 400 \times 300 \times 6$	θ	Node
Soft Q network size	$18 \times 800 \times 500 \times 400 \times 400 \times 300 \times 1$	$\phi_{1,2}$	Node
Soft value network size	$12 \times 800 \times 500 \times 400 \times 400 \times 300 \times 1$	ψ, $\bar{\psi}$	Node
LSTM network size	$[3,5] \times$ LSTM(800) $\times$ LSTM(400) $\times$ LSTM(200) $\times 100 \times 50 \times 9$		Node
Learning rate	0.0001		
Replay memory size	10^6	$\mathcal{D}$	Buffer
Episode maximum step	50	T	Step
Soft value target copy rate	0.005	τ	
Mini batch size	512	m	Batch
Environment noise deviation	0.002	ϵ_e	
Action step size	0.3813	α	
Goal boundary	0.2	η	
Dicount factor	0.99	γ	
Entropy temperature parameter	0.2	β	

4.2. Experiment

In this subsection, the performance of the proposed SAC-based path planning is tested using two real robot manipulators. For the moving obstacle in the experiment, it is assumed that there is one moving obstacle that moves back and forth along a horizontal line segment. It is also assumed that the obstacle moves with a random speed belonging to a small interval. Figure 14 shows the experimental setup.

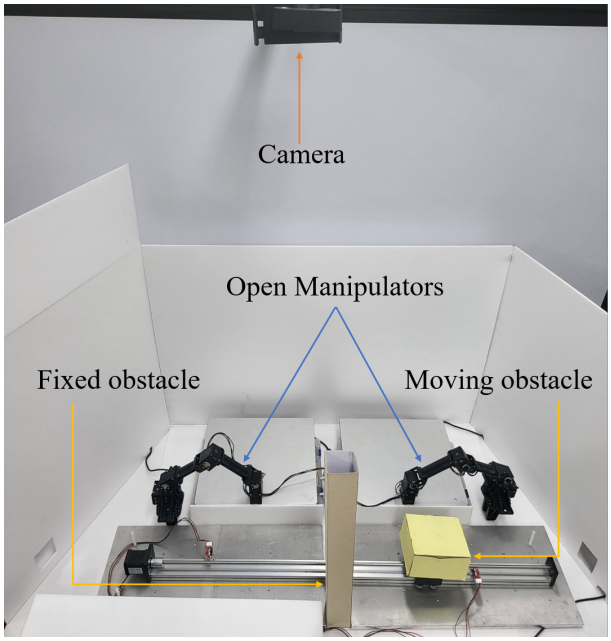


Figure 14. The experimental setup using two open manipulators and obstacles.

In applying the proposed SAC-based path planning to the real robot, the training procedure is exactly the same as the simulation-based case study if the moving obstacles behavior can be generated artificially and the LSTM is trained using the artificially generated data. If it is not the case, the time series data for the moving obstacles can be measured independently from the real moving obstacle as explained before. Figure 15 shows the training results of the LSTM. In light of the convergence of the loss function, it is confirmed that the LSTM is successfully trained.

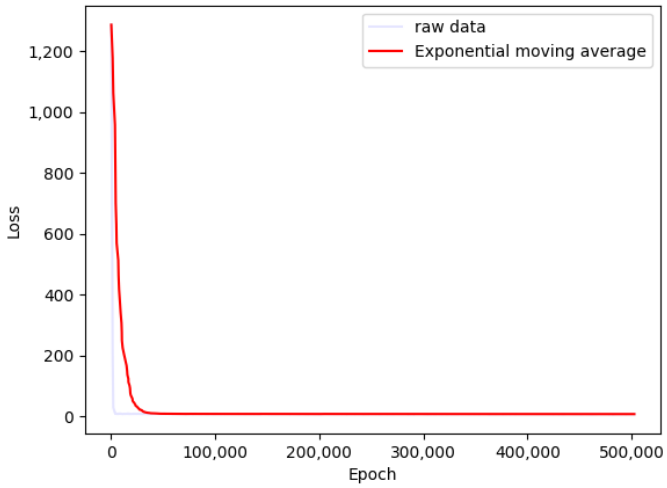


Figure 15. Loss function of experiment LSTM training.

In the case of training the SAC-based path planning agent, the entire procedure is exactly the same as in the simulation-based case study. Figures 11 and 12 show the success ratio and reward history, respectively. The graph shows faster learning ability and convergence than the results in Figures 7 and 8 because we learned in an obstacle environment that moves only on the y-axis according to the experiment. From the success ratio and reward, it can be seen that the SAC-based path planning agent's learning ends successfully.

With the trained SAC-based path planning agent, the experiment for the testing in Figure 4 is carried out with arbitrarily chosen starting and ending points in the workspace.

The test results can be found in Figure 16. The success ratio converges to 100% quickly. It can also be confirmed from the video on the web page https://url.kr/4m562t (accessed on 25 September 2022). Figure 13b shows the prediction performance of the trained LSTM in the middle of the testing. From the testing results, it is confirmed that the proposed SAC-based path planning agent successfully computes the optimal path for the arbitrary starting and goal points in the workspace using the LSTM-based prediction of the moving obstacle.

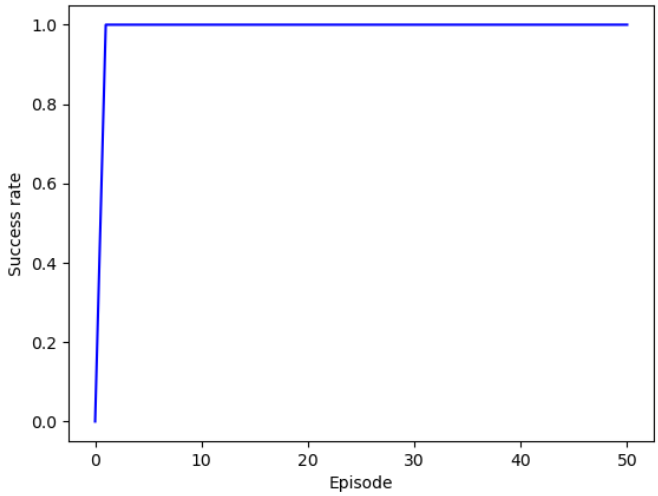


Figure 16. Experiment: success rate of the proposed path planning in testing.

5. Summary and Conclusions

This paper proposes a deep reinforcement learning-based path planning algorithm using the LSTM for multi-arm manipulators when there are both fixed and moving obstacles. Since the MDP under consideration is a high-dimensional problem with a continuous action space, soft actor-critic (SAC) is employed for reinforcement learning. In order to use the SAC for the path planning problem under consideration, MDP (Markov decision process) is defined appropriately first. Then, the LSTM network is designed such that it can predict the future position of the moving obstacle using current and past positions. With the trained LSTM network, the SAC-based path planning agent is devised such that the prediction of the future obstacle position from the trained LSTM is also used as the input to the neural network for the SAC. Then, the SAC path planning agent computes the optimal path connecting arbitrary starting and goal points in the workspace. The proposed SAC-based path planning algorithm is tested in both simulation and experiment. In both cases, the proposed planning computes the optimal path successfully when there are both fixed and moving obstacles. To be specific, Figures 9 and 11 show that the success ratio of path generation for arbitrary starting and goal points converges to 100%. Additionally, it is confirmed that the LSTM predicts the future position of the obstacle.

Author Contributions: K.-W.P. and M.K. surveyed the backgrounds of this research, designed the deep learning network, and performed the simulations and experiments to show the benefits of the proposed method. J.-S.K. and J.-H.P. supervised and supported this study. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (NRF-2019R1F1A1061197, NRF-2019R1A6A1A03032119).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

MAMMDP	Multi-Arm Manipulator Markov Decision Process
SAC	Soft Actor-Critic
HER	Hindsight Experience Replay
PRM	Probabilistic Road Map
RRT	Rapid exploring Random Trees
MDP	Markov Decision Process
OBB	Oriented Bounding Boxes
LSTM	Long Short-Term Memory

References

1. Berman, S.; Schechtman, E.; Edan, Y. Evaluation of automatic guided vehicle systems. *Robot. Comput.-Integr. Manuf.* **2009**, *25*, 522–528. [CrossRef]
2. Evjemo, L.D.; Gjerstad, T.; Grøtli, E.I.; Sziebig, G. Trends in smart manufacturing: Role of humans and industrial robots in smart factories. *Curr. Robot. Rep.* **2020**, *1*, 35–41. [CrossRef]
3. Arents, J.; Abolins, V.; Judvaitis, J.; Vismanis, O.; Oraby, A.; Ozols, K. Human–robot collaboration trends and safety aspects: A systematic review. *J. Sens. Actuator Netw.* **2021**, *10*, 48. [CrossRef]
4. Spong, M.; Hutchinson, S.; Vidyasagar, M. *Robot Modeling and Control*; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2006; Volume 26.
5. Latombe, J.C. *Robot Motion Planning*; Kluwer Academic Publishers: Boston, MA, USA, 1991.
6. Buhl, J.F.; Grønhøj, R.; Jørgensen, J.K.; Mateus, G.; Pinto, D.; Sørensen, J.K.; Bøgh, S.; Chrysostomou, D. A dual-arm collaborative robot system for the smart factories of the future. *Procedia Manuf.* **2019**, *38*, 333–340. [CrossRef]
7. Bonci, A.; Cen Cheng, P.D.; Indri, M.; Nabissi, G.; Sibona, F. Human-robot perception in industrial environments: A survey. *Sensors* **2021**, *21*, 1571. [CrossRef]
8. Pendleton, S.; Andersen, H.; Du, X.; Shen, X.; Meghjani, M.; Eng, Y.; Rus, D.; Ang, M. Perception, planning, control, and coordination for autonomous vehicles. *Machines* **2017**, *5*, 6. [CrossRef]
9. Le, C.H.; Le, D.T.; Arey, D.; Gheorghe, P.; Chu, A.M.; Duong, X.B.; Nguyen, T.T.; Truong, T.T.; Prakash, C.; Zhao, S.T.; et al. Challenges and conceptual framework to develop heavy-load manipulators for smart factories. *Int. J. Mechatronics Appl. Mech.* **2020**, *8*, 209–216.
10. Arents, J.; Greitans, M.; Lesser, B. Construction of a smart vision-guided robot system for manipulation in a dynamic environment. In *Artificial Intelligence for Digitising Industry*; River Publishers: Gistrup, Denmark, 2022; pp. 205–220.
11. Hart, P.E.; Nilsson, N.J.; Raphael, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **1968**, *4*, 100–107. [CrossRef]
12. Karaman, S.; Frazzoli, E. Sampling-based algorithms for optimal motion planning. *Int. J. Robot. Res.* **2011**, *30*, 846–894. [CrossRef]
13. Kavraki, L.E.; Svestka, P.; Latombe, J.C.; Overmars, M.H. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Trans. Robot. Autom.* **1996**, *12*, 566–580. [CrossRef]
14. Zhang, H.Y.; Lin, W.M.; Chen, A.X. Path planning for the mobile robot: A review. *Symmetry* **2018**, *10*, 450. [CrossRef]
15. Schrijver, A. *Combinatorial Optimization: Polyhedra and Efficiency*; Springer: Berlin/Heidelberg, Germany, 2003; Volume 24.
16. Kuffner, J.J.; LaValle, S.M. RRT-connect: An efficient approach to single-query path planning. In Proceedings of the 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No. 00CH37065), San Francisco, CA, USA, 24–28 April 2000; Volume 2, pp. 995–1001.
17. Davis, L. *Handbook of Genetic Algorithms*; CumInCAD: Ljubljana, Slovenia, 1991.
18. Dorigo, M.; Birattari, M.; Stutzle, T. Ant colony optimization. *IEEE Comput. Intell. Mag.* **2006**, *1*, 28–39. [CrossRef]
19. Kennedy, J.; Eberhart, R. Particle swarm optimization. In Proceedings of the ICNN'95-International Conference on Neural Networks, Perth, WA, Australia, 27 November–1 December 1995; Volume 4, pp. 1942–1948.
20. Bertsimas, D.; Tsitsiklis, J. Simulated annealing. *Stat. Sci.* **1993**, *8*, 10–15. [CrossRef]
21. Sangiovanni, B.; Rendiniello, A.; Incremona, G.P.; Ferrara, A.; Piastra, M. Deep reinforcement learning for collision avoidance of robotic manipulators. In Proceedings of the 2018 European Control Conference (ECC), Limassol, Cyprus, 12–15 June 2018; pp. 2063–2068.
22. Prianto, E.; Park, J.H.; Bae, J.H.; Kim, J.S. Deep reinforcement learning-based path planning for multi-arm manipulators with periodically moving obstacles. *Appl. Sci.* **2021**, *11*, 2587. [CrossRef]
23. Zhong, J.; Wang, T.; Cheng, L. Collision-free path planning for welding manipulator via hybrid algorithm of deep reinforcement learning and inverse kinematics. *Complex Intell. Syst.* **2022**, *8*, 1899–1912. [CrossRef]
24. Xie, R.; Meng, Z.; Wang, L.; Li, H.; Wang, K.; Wu, Z. Unmanned aerial vehicle path planning algorithm based on deep reinforcement learning in large-scale and dynamic environments. *IEEE Access* **2021**, *9*, 24884–24900. [CrossRef]

25. Choset, H.M.; Hutchinson, S.; Lynch, K.M.; Kantor, G.; Burgard, W.; Kavraki, L.E.; Thrun, S.; Arkin, R.C. *Principles of Robot Motion: Theory, Algorithms, and Implementation*; MIT Press: Cambridge, MA, USA, 2005.
26. Lozano-Perez. Spatial planning: A configuration space approach. *IEEE Trans. Comput.* **1983**, *C-32*, 108–120. [CrossRef]
27. Laumond, J.P.P. *Robot Motion Planning and Control*; Springer: Berlin/Heidelberg, Gemany, 1998.
28. Bergen, G.V.D.; Bergen, G.J. *Collision Detection*, 1st ed.; Morgan Kaufmann Publishers Inc.: San Francisco, CA, USA, 2003.
29. Bergen, G.v.d. Efficient collision detection of complex deformable models using AABB trees. *J. Graph. Tools* **1997**, *2*, 1–13. [CrossRef]
30. Ericson, C. *Real-Time Collision Detection*; CRC Press, Inc.: Boca Raton, FL, USA, 2004.
31. Fares, C.; Hamam, Y. Collision detection for rigid bodies: A state of the art review. In Proceedings of the GraphiCon 2005—International Conference on Computer Graphics and Vision, Proceedings, Novosibirsk Akademgorodok, Russia, 20–24 June 2005.
32. Puterman, M.L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, 1st ed.; John Wiley & Sons, Inc.: Hoboken, NJ, USA, 1994.
33. Sutton, R.S.; Barto, A.G. *Reinforcement Learning: An Introduction*; A Bradford Book: Cambridge, MA, USA, 2018.
34. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A.K.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [CrossRef]
35. Hausknecht, M.; Stone, P. Deep recurrent q-learning for partially observable mdps. In Proceedings of the 2015 AAAI Fall Symposium Series, Arlington, VA, USA, 12–14 November 2015.
36. Sutton, R.S.; McAllester, D.; Singh, S.; Mansour, Y. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In Proceedings of the 12th International Conference on Neural Information Processing Systems, Cambridge, MA, USA, 1999; pp. 1057–1063.
37. Silver, D.; Lever, G.; Heess, N.; Degris, T.; Wierstra, D.; Riedmiller, M. Deterministic Policy Gradient Algorithms. In Proceedings of the 31st International Conference on International Conference on Machine Learning, Beijing, China, 21–26 June 2014; Volume 32, pp. 387–395.
38. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous control with deep reinforcement learning. In Proceedings of the ICLR (Poster), San Juan, Puerto Rico, 2–4 May 2016.
39. Mnih, V.; Badia, A.P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Proceedings of the International Conference on Machine Learning. PMLR, New York, NY, USA, 20–22 June 2016; pp. 1928–1937.
40. Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.; Moritz, P. Trust region policy optimization. In Proceedings of the International Conference on Machine Learning. PMLR, Lille, France, 7–9 July 2015; pp. 1889–1897.
41. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv* **2017**, arXiv:1707.06347.
42. Abdolmaleki, A.; Springenberg, J.T.; Tassa, Y.; Munos, R.; Heess, N.; Riedmiller, M. Maximum a Posteriori Policy Optimisation. In Proceedings of the International Conference on Learning Representations, Vancouver, BC, Canada, 30 April–3 May 2018.
43. Barth-Maron, G.; Hoffman, M.W.; Budden, D.; Dabney, W.; Horgan, D.; Dhruva, T.; Muldal, A.; Heess, N.; Lillicrap, T. Distributed Distributional Deterministic Policy Gradients. In Proceedings of the International Conference on Learning Representations, Vancouver, BC, Canada, 30 April–3 May 2018.
44. Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized experience replay. *arXiv* **2015**, arXiv:1511.05952.
45. Andrychowicz, M.; Wolski, F.; Ray, A.; Schneider, J.; Fong, R.; Welinder, P.; McGrew, B.; Tobin, J.; Pieter Abbeel, O.; Zaremba, W. Hindsight Experience Replay. In Proceedings of the Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 4–9 December 2017; pp. 5048–5058.
46. Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft actor-Critic: Off-Policy maximum entropy deep reinforcement learning with a stochastic actor. In Proceedings of the International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; pp. 1861–1870.
47. Abdel-Nasser, M.; Mahmoud, K. Accurate photovoltaic power forecasting models using deep LSTM-RNN. *Neural Comput. Appl.* **2019**, *31*, 2727–2740. [CrossRef]
48. Gensler, A.; Henze, J.; Sick, B.; Raabe, N. Deep Learning for solar power forecasting—An approach using AutoEncoder and LSTM Neural Networks. In Proceedings of the 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Budapest, Hungary, 9–12 October 2016; pp. 002858–002865.
49. Ghosh, S.; Vinyals, O.; Strope, B.; Roy, S.; Dean, T.; Heck, L. Contextual lstm (clstm) models for large scale nlp tasks. *arXiv* **2016**, arXiv:1602.06291.
50. Melamud, O.; Goldberger, J.; Dagan, I. context2vec: Learning generic context embedding with bidirectional lstm. In Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning, Berlin, Germany, 11–12 August 2016; pp. 51–61.
51. Choset, H.; Lynch, K.; Hutchinson, S.; Kantor, G.; Burgard, W. *Principles of Robot Motion: Theory, Algorithms, and Implementations*; Intelligent Robotics and Autonomous Agents Series; MIT Press: Cambridge, MA, USA, 2005.
52. Latombe, J.C. *Robot Motion Planning*; Springer Science & Business Media: Berlin/Heidelberg, Germany, 2012; Volume 124.
53. Fujimoto, S.; Hoof, H.; Meger, D. Addressing function approximation error in actor-critic methods. In Proceedings of the International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018; pp. 1582–1591.